Московский авиационный институт
(национальный исследовательский университет)

Факультет компьютерных наук и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №4 по курсу «Дискретный анализ»

Студент:	М. Г. Григорьев
Преподаватель:	С. А. Михайлова
Группа:	М8О-201БВ
Дата:	
Оценка:	
Подпись:	

Москва, 2026

Лабораторная работа №4

Задача: Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск большого количества образцов при помощи алгоритма Ахо–Корасик.

Вариант алфавита: Числа в диапазоне от 0 до $2^{32} - 1$.

Искомые образцы задаются по одному на строку до первой пустой строки. После неё следует текст, состоящий из слов или чисел. Для каждого найденного вхождения требуется вывести номер строки текста, номер слова в строке, с которого начинается вхождение, и порядковый номер образца. Нумерация строк, слов и образцов начинается с единицы.

1 Описание

В работе реализован алгоритм Ахо–Корасик для поиска множества образцов в тексте. Алгоритм сначала строит trie по всем образцам, затем дополняет его связями неудачи и выходными ссылками. После этого текст обрабатывается за один проход: каждое очередное число переводит автомат в новое состояние, а все образцы, заканчивающиеся в этом состоянии, сразу выводятся как найденные [**?**, **?**].

Особенность варианта состоит в большом алфавите: символами являются не буквы, а числа типа `uint32_t`. Поэтому переходы из вершины нельзя хранить в массиве размера 2^{32}. В реализации используется разреженное хранение переходов:

```
std::unordered_map<std::uint32_t, std::size_t>
```

Такой подход хранит только реально существующие рёбра trie и соответствует требуемому алфавиту. Записи вида `0002`, `02` и `2` при чтении превращаются в одно и то же число.

1 Структура автомата

Вершины автомата хранятся в массиве `nodes_`. Корень имеет индекс 0. Каждая вершина содержит:

- `next` – переходы по числам алфавита;
- `failure_link` – связь неудачи, то есть переход к наибольшему собственному суффиксу текущего состояния;
- `output_link` – ссылка на ближайшую вершину по цепочке связей неудачи, в которой заканчивается образец;
- `patterns` – список образцов, которые заканчиваются непосредственно в данной вершине.

Для найденного образца хранится его номер во входе и длина. Длина нужна не для работы автомата, а для восстановления позиции начала вхождения в тексте.

2 Добавление образцов

Метод `AddPattern` добавляет один образец в trie. Обход начинается из корня. Для каждого числа образца проверяется наличие перехода из текущей вершины. Если переход отсутствует, создаётся новая вершина; если переход уже существует, обход

продолжается по нему. После обработки последнего числа в текущей вершине сохраняется информация о завершившемся образце: его номер и длина.

Также при добавлении поддерживается значение `max_pattern_length_`. Оно равно максимальной длине среди всех образцов и далее используется для хранения только нужного числа последних позиций текста.

3 Построение связей

Метод `BuildLinks` строит связи неудачи обходом в ширину. Для детей корня связь неудачи ведёт в корень. Для каждой другой вершины рассматривается ребро из родителя по некоторому числу `value`. Алгоритм берёт связь неудачи родителя и поднимается по цепочке связей неудачи, пока не найдёт вершину, из которой существует переход по `value`. Если такой переход найден, связь неудачи текущего ребёнка указывает на соответствующую вершину; иначе она указывает на корень.

После связи неудачи для вершины строится `output_link`. Если вершина по связи неудачи сама содержит завершение образца, выходная ссылка ведёт в неё. Иначе используется выходная ссылка этой вершины. Это позволяет быстро находить все образцы, являющиеся суффиксами текущей прочитанной последовательности.

4 Переходы при поиске

Метод `NextState` выполняет переход автомата по очередному числу текста. Если из текущей вершины есть переход по этому числу, автомат сразу переходит по нему. Если перехода нет, выполняется движение по `failure_link`, пока переход не будет найден или пока автомат не вернётся в корень. Если перехода нет даже из корня, состояние остаётся равным нулю.

Метод `ForEachMatch` проверяет все образцы, которые заканчиваются в текущем состоянии. Сначала рассматриваются образцы самой вершины, затем выполняются переходы по `output_link`. Для каждого найденного образца вызывается переданная callback-функция, которая в `main` печатает ответ.

5 Чтение входа и восстановление позиции

Функция `ParseNumbers` разбирает одну строку на числа и для каждого числа вызывает переданный обработчик. Такой формат позволяет использовать один и тот же парсер и для строк с образцами, и для строк текста.

Во время обработки текста автомат находит момент окончания образца. В ответе же требуется вывести позицию начала. Для этого используется очередь `recent_positions`, где хранятся позиции последних прочитанных чисел. Достаточно хранить не больше

`max_pattern_length` позиций: более старое число не может быть началом только что найденного образца.

Если найден образец длины k, то его начало находится на k-й позиции с конца очереди:

`recent_positions.size() - pattern.length`

Полученная позиция выводится вместе с номером найденного образца.

Асимптотика реализации:

- построение trie – $O(P)$, где P – суммарная длина всех образцов;
- построение связей – $O(P)$ в среднем при использовании `unordered_map`;
- поиск – $O(T + Ans)$, где T – число чисел в тексте, Ans – число найденных вхождений;
- память – $O(P)$ на автомат и $O(L)$ на очередь позиций, где L – максимальная длина образца.

2 Исходный код

Ниже приведены основные фрагменты реализации.

Листинг 1: Структуры позиций, образцов и вершины автомата

```
struct Position {
    std::size_t line;
    std::size_t word;
};

struct PatternInfo {
    std::size_t id;
    std::size_t length;
};

class AhoCorasick {
private:
    struct Node {
        std::unordered_map<std::uint32_t, std::size_t> next;
        std::size_t failure_link = 0;
        int output_link = -1;
        std::vector<PatternInfo> patterns;
    };
```

Листинг 2: Добавление образца в trie

```
void AddPattern(const std::vector<std::uint32_t>& pattern, std::size_t pattern_id) {
    std::size_t current = 0;

    for (std::uint32_t value : pattern) {
        auto found = nodes_[current].next.find(value);
        if (found == nodes_[current].next.end()) {
            const std::size_t new_node = nodes_.size();
            nodes_.push_back(Node());
            nodes_[current].next[value] = new_node;
            current = new_node;
        } else
            current = found->second;
    }

    nodes_[current].patterns.push_back(PatternInfo{pattern_id, pattern.size()});
    if (pattern.size() > max_pattern_length_)
        max_pattern_length_ = pattern.size();
}
```

Листинг 3: Построение связей неудачи и выходных ссылок

```
void BuildLinks() {
    std::queue<std::size_t> queue;

    for (const auto& edge : nodes_[0].next) {
        const std::size_t child = edge.second;
        nodes_[child].failure_link = 0;
        nodes_[child].output_link = -1;
        queue.push(child);
    }

    while (!queue.empty()) {
        const std::size_t vertex = queue.front();
        queue.pop();

        for (const auto& edge : nodes_[vertex].next) {
            const std::uint32_t value = edge.first;
            const std::size_t child = edge.second;
            std::size_t fallback = nodes_[vertex].failure_link;

            while (fallback != 0 && nodes_[fallback].next.find(value) == nodes_[fallback].next.end())
                fallback = nodes_[fallback].failure_link;

            auto transition = nodes_[fallback].next.find(value);
            if (transition != nodes_[fallback].next.end() && transition->second !=
```

```
                    child)
                    nodes_[child].failure_link = transition->second;
                else
                    nodes_[child].failure_link = 0;

                const std::size_t suffix = nodes_[child].failure_link;
                if (!nodes_[suffix].patterns.empty())
                    nodes_[child].output_link = static_cast<int>(suffix);
                else
                    nodes_[child].output_link = nodes_[suffix].output_link;

                queue.push(child);
            }
        }
    }
```

Листинг 4: Переход автомата и перечисление найденных образцов

```
    std::size_t NextState(std::size_t state, std::uint32_t value) const {
        while (state != 0 && nodes_[state].next.find(value) == nodes_[state].next.end()
            )
            state = nodes_[state].failure_link;

        auto transition = nodes_[state].next.find(value);
        if (transition != nodes_[state].next.end())
            return transition->second;

        return 0;
    }

    template <typename Callback>
    void ForEachMatch(std::size_t state, Callback on_match) const {
        int current = static_cast<int>(state);

        while (current != -1) {
            for (const PatternInfo& pattern : nodes_[current].patterns)
                on_match(pattern);

            current = nodes_[current].output_link;
        }
    }
```

Листинг 5: Разбор ввода и потоковая обработка текста

```
template <typename Callback>
void ParseNumbers(const std::string& line, Callback on_number) {
    std::size_t index = 0;

```

```
    while (index < line.size()) {
        while (index < line.size() &&
               std::isspace(static_cast<unsigned char>(line[index])))
            ++index;

        if (index == line.size())
            break;

        std::uint64_t value = 0;
        while (index < line.size() &&
               !std::isspace(static_cast<unsigned char>(line[index]))) {
            value = value * 10 + static_cast<std::uint64_t>(line[index] - '0');
            ++index;
        }

        on_number(static_cast<std::uint32_t>(value));
    }
}

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    AhoCorasick automaton;
    std::string line;
    std::size_t pattern_id = 1;

    while (std::getline(std::cin, line)) {
        std::vector<std::uint32_t> pattern;
        ParseNumbers(line, [&pattern](std::uint32_t value) {
            pattern.push_back(value);
        });

        if (pattern.empty())
            break;

        automaton.AddPattern(pattern, pattern_id);
        ++pattern_id;
    }

    if (pattern_id == 1)
        return 0;

    automaton.BuildLinks();

    std::deque<Position> recent_positions;
    std::size_t state = 0;
    std::size_t line_number = 0;
    const std::size_t max_pattern_length = automaton.MaxPatternLength();
```

```

    while (std::getline(std::cin, line)) {
        ++line_number;
        std::size_t word_number = 0;

        ParseNumbers(line, [&](std::uint32_t value) {
            ++word_number;

            recent_positions.push_back(Position{line_number, word_number});
            if (recent_positions.size() > max_pattern_length)
                recent_positions.pop_front();

            state = automaton.NextState(state, value);

            automaton.ForEachMatch(state, [&](const PatternInfo& pattern) {
                const std::size_t start_index = recent_positions.size() - pattern.length;
                const Position& start = recent_positions[start_index];
                std::cout << start.line << ", " << start.word << ", " << pattern.id << '\n';
            });
        });
    }

    return 0;
}
```

3 Консоль

```
$ g++ -std=c++14 -O2 -Wall -Wextra -pedantic main.cpp -o lab4.out
$ ./lab4.out
1 0002 001 2
1 2 1
2 02 2 2

01 2 1 2   1 2 1 3
2 2 2 2 02
1,1,2
1,1,1
1,3,2
1,3,1
1,5,2
2,1,3
2,2,3
```

Порядок вывода может отличаться от порядка в примере из условия, поскольку в условии явно указано, что порядок следования найденных вхождений несущественен.

4 Тестирование и бенчмарк

Корректность программы проверялась на примере из условия, на граничных случаях и на случайных малых тестах. Для случайных тестов результат сравнивался с прямым перебором всех позиций текста и всех образцов. Отдельно проверялись следующие ситуации:

- образец длины 1;
- несколько образцов, заканчивающихся в одной позиции текста;
- образцы, являющиеся суффиксами других образцов;
- числа с ведущими нулями во входе;
- значение $2^{32} - 1$;
- пустые строки в текстовой части входа.

Для оценки производительности был подготовлен отдельный бенчмарк. Данные генерируются скриптом `gen_benchmark_data.py`, а измерения выполняются программой `benchmark.cpp`. Генерация входа и разбор данных выполняются вне измеряемого участка. Время замеряется отдельно для построения автомата и для прохода по уже сохранённому в памяти тексту. Для каждого случая выполнено 5 запусков, в таблицах приведены медианные значения. Полные результаты сохранены в файле `results/benchmark.csv`.

Команды запуска:

```
$ g++ -std=c++14 -O2 -Wall -Wextra -pedantic main.cpp -o lab4.out
$ bash run_benchmarks.sh
```

1 Сравнение с наивным поиском

На малых и средних тестах алгоритм Ахо–Корасик сравнивался с прямым перебором всех позиций текста и всех образцов. В последнем столбце указано отношение `Naive / AC`; значение больше 1 означает, что автомат работает быстрее. Для всех строк число найденных вхождений у двух алгоритмов совпало.

Образцов	Длина текста	AC build, μs	AC search, μs	Naive, μs	Naive / AC
10	20 000	7	189	238	1.21×
50	20 000	11	110	543	4.49×
100	20 000	19	93	1128	10.16×

Даже на небольшом тексте преимущество автомата быстро растёт с увеличением числа образцов. Это ожидаемо: наивный алгоритм заново сравнивает каждый образец с каждой позицией текста, а Ахо–Корасик после построения автомата выполняет один общий проход.

2 Масштабирование по длине текста

В следующей серии число образцов фиксировано и равно 500, длина каждого образца равна 5. Менялась только длина текста. Наивный алгоритм здесь не запускался, так как его стоимость уже существенно выше и не нужна для проверки масштабирования автомата.

Чисел в тексте	Вхождений	Build, μs	Search, μs	Total, μs
50 000	50	99	266	363
100 000	100	100	534	633
500 000	500	103	2686	2794
1 000 000	1000	102	5455	5575

Время построения практически не меняется, потому что набор образцов один и тот же. Время поиска растёт почти линейно относительно числа чисел в тексте: при увеличении текста с 100000 до 1000000 время прохода выросло примерно в 10.2 раза.

3 Масштабирование по числу образцов

В этой серии длина текста фиксирована и равна 200000 чисел, длина каждого образца равна 5. Менялось число образцов, то есть суммарная длина trie.

Образцов	Суммарная длина	Build, μs	Search, μs	Total, μs
100	500	19	1292	1317
500	2500	105	1102	1202
1000	5000	197	1102	1301
5000	25000	984	1066	2050

Время построения растёт вместе с суммарной длиной образцов, что соответствует оценке $O(P)$. Время поиска при этом остаётся одного порядка: текст имеет ту же длину, а сгенерированные переходы хранятся разреженно в `unordered_map`. Различия между строками связаны с константами хеш-таблиц и конкретным набором переходов.

4 Нагрузка на выходные ссылки

Отдельно проверялся случай с большим числом найденных ответов. Для этого использовались образцы x, $x\,x$, …, x^k, а текст состоял из 100000 одинаковых чисел x. В таком тесте почти в каждой позиции заканчивается сразу несколько образцов.

Макс. длина образца	Найдено вхождений	Search, μs	Total, μs
5	499 990	470	471
10	999 955	821	823
20	1 999 810	2210	2216

Этот тест показывает вклад выходных ссылок и количества ответов. При одинаковой длине текста время увеличивается вместе с числом найденных вхождений, что соответствует оценке поиска $O(T + Ans)$.

5 Выводы

В лабораторной работе реализован алгоритм Ахо–Корасик над числовым алфавитом `uint32_t`. Главная особенность варианта в том, что переходы автомата строятся по значениям чисел, а не по символам их строкового представления. Поэтому записи с ведущими нулями обрабатываются как те же самые числа, и решение не сводит алфавит к десятичным символам.

Наиболее важной частью реализации стали связи неудачи и выходные ссылки. Связи неудачи позволяют не начинать поиск заново после несовпадения, а выходные ссылки нужны для случаев, когда несколько образцов заканчиваются в одной позиции текста. Без них образцы, являющиеся суффиксами более длинных, можно было бы пропустить.

Для восстановления позиции начала вхождения программа хранит только последние `max_pattern_length` позиций текста. Поэтому текст не нужно сохранять целиком: автомат обрабатывает вход потоково, а память на позиции зависит от максимальной длины образца, а не от размера всего текста.

Бенчмарк подтвердил ожидаемое поведение. На тесте из 100 образцов и 20000 чисел текста Ахо–Корасик отработал за 111 мкс против 1128 мкс у наивного перебора, то есть примерно в $10.16\times$ быстрее. При фиксированных 500 образцах проход по 1000000 чисел занял 5455 мкс, что показывает близкий к линейному рост времени поиска от длины текста. Отдельный сценарий с вложенными образцами дал 1999810 найденных вхождений на 100000 числах, и рост времени в нём соответствует оценке $O(T + Ans)$.

Итоговая реализация соответствует условию задачи и на практике показывает главное преимущество Ахо–Корасик: множество образцов ищется одним проходом по тексту после предварительного построения автомата.

Список литературы

[1] Alfred V. Aho, Margaret J. Corasick. *Efficient String Matching: An Aid to Bibliographic Search.* Communications of the ACM, 1975.

[2] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 3-е издание.* — М.: Вильямс.